

IMPLEMENTING USING .NET

- bin/ → Generated compiled files; no need to edit manually.
- obj/ → Temporary/intermediate build files generated by .NET.
- bin/ and obj/ are automatically recreated when the project is built/run.
- Controllers/ → Handles application requests/routes.
- Models/ → Defines application data/entities.
- Data/ → Database and Entity Framework Core configuration.
- Views/ → HTML/Razor UI pages.
- wwwroot/ → CSS, JavaScript, images, and other static files.
- PlacementTracker.Tests/ → Automated unit/integration tests.
- Properties/ → Project/run configuration.
- .csproj → Main project file containing dependencies and .NET configuration.
- Microsoft.*.dll, SQLitePCLRaw.*.dll, etc. → External .NET/EF Core/SQLite libraries required by the application.
- PlacementTracker.dll → Compiled version of your application.
- PlacementTracker.pdb → Debugging information.
- bin/ and obj/ can safely be deleted; .NET recreates them automatically.
- Add bin/ and obj/ to .gitignore so they aren't uploaded to GitHub.

ApplicationsController.cs

Byte 1: What the Controller Does

Builds on: None — starting point

In plain terms:

ApplicationsController is the part of the application that handles requests related to placement applications. It connects the browser, database, and HTML views.

The code:

```
public class ApplicationsController : Controller
{
    private readonly ApplicationDbContext _db;
}
```

What's happening:

The controller inherits from ASP.NET Core's `Controller` class, giving it access to MVC features such as returning views and redirects. `_db` represents the database connection/context used to read and modify applications.

Why it matters:

Without this controller, the application would have no central place to process routes such as creating, editing, viewing, or deleting placement applications.

Byte 2: Connecting the Database

Builds on: Byte 1

In plain terms:

The controller receives the database context through its constructor. This is called **dependency injection** — ASP.NET provides the object instead of the controller creating it manually.

The code:

```
public ApplicationsController(ApplicationDbContext db)
{
    _db = db;
}
```

What's happening:

ASP.NET creates `ApplicationDbContext` and passes it into the controller. The controller stores it in `_db`, which is then used for database operations.

Why it matters:

It keeps database logic organized and makes the controller easier to test and maintain.

Byte 3: Normalizing Form Input

Builds on: Byte 2

In plain terms:

HTML forms can submit empty strings when the user leaves optional fields blank. This method converts those empty values into null so the application treats them as genuinely missing.

The code:

```
private void NormalizeOptionalFields(models.Application app)
```

```

{
    if (string.IsNullOrEmpty(app.JobUrl))
    {
        app.JobUrl = null;
        ModelState.Remove(nameof(models.Application.JobUrl));
    }

    if (string.IsNullOrEmpty(app.Notes))
    {
        app.Notes = null;
        ModelState.Remove(nameof(models.Application.Notes));
    }
}

```

What's happening:

IsNullOrEmpty() checks whether a value is empty or contains only spaces. Empty JobUrl and Notes values become null. Their previous validation errors are also removed because an empty optional field should not fail URL validation.

Why it matters:

It prevents optional fields from incorrectly causing validation failures.

Byte 4: Custom 404 Page

Builds on: Byte 1

In plain terms:

When a user requests an application ID that doesn't exist, the controller returns the project's custom 404 page instead of continuing with a missing object.

The code:

```

private IActionResult CustomNotFound()
{
    Response.StatusCode = 404;
    return View("~/Views/Shared/404.cshtml");
}

```

What's happening:

The response status is explicitly changed to HTTP 404, meaning "Not Found", and the custom 404.cshtml page is displayed.

Why it matters:

Users get a proper error page instead of an exception or broken screen when an application doesn't exist.

Byte 5: Displaying All Applications

Builds on: Bytes 1–2

In plain terms:

The Index() action handles the home page. It retrieves all placement applications from SQLite through Entity Framework Core and sends them to the view.

The code:

```
[HttpGet("/")]
public async Task<IActionResult> Index()
{
    var apps = await _db.Applications
        .OrderByDescending(a => a.Id)
        .ToListAsync();

    return View("Index", apps);
}
```

What's happening:

GET / triggers Index(). The database records are sorted by ID in descending order, so newer applications appear first. ToListAsync() executes the database query asynchronously.

Why it matters:

This is the **Read** part of CRUD — it retrieves stored placement applications and displays them.

Byte 6: Showing the Create Form

Builds on: Byte 5

In plain terms:

Before creating an application, the user needs a form. `New()` prepares a default application object and sends it to the Create view.

The code:

```
[HttpGet("/applications/new")]
public IActionResult New()
{
    var model = new Models.Application
    {
        Status = "Applied",
        AppliedOn = DateTime.Today
    };

    return View("Create", model);
}
```

What's happening:

When `/applications/new` is opened, a new application object is created with sensible default values. The Create view uses this object to pre-fill the form.

Why it matters:

It gives the user a ready-to-use form instead of making them enter every default value manually.

Byte 7: Creating and Saving an Application

Builds on: Bytes 3 and 6

In plain terms:

`Create()` processes the form submitted by the user. It normalizes the input, validates it, saves it to the database if valid, and then redirects to the application's details page.

The code:

```
[HttpPost("/applications")]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Create(Models.Application application)
{
    NormalizeOptionalFields(application);
```

```

        application.CreatedAt = DateTime.UtcNow;

        if (!ModelState.IsValid)
        {
            return View("Create", application);
        }

        _db.Applications.Add(application);
        await _db.SaveChangesAsync();

        return RedirectToAction(nameof(Details),
            new { id = application.Id });
    }

```

What's happening:

The submitted form is automatically converted into an Application object. Validation happens through ModelState. If validation fails, the form is shown again with the entered values. If valid, Entity Framework adds the object and SaveChangesAsync() permanently stores it.

Why it matters:

This implements the **Create** part of CRUD while preventing invalid data from reaching the database.

Byte 8: Viewing One Application

Builds on: Byte 7

In plain terms:

Details() retrieves one application using its ID and displays it. If that ID doesn't exist, the custom 404 page is returned.

The code:

```

[HttpGet("/applications/{id:int}")]
public async Task<IActionResult> Details(int id)
{
    var app = await _db.Applications.FindAsync(id);

```

```

        if (app is null)
        {
            return CustomNotFound();
        }

        return View("Details", app);
    }

```

What's happening:

The {id:int} part means the route expects an integer ID. FindAsync(id) searches the database for that application. The result is passed to the Details view.

Why it matters:

This provides the **Read** operation for a single placement application.

Byte 9: Loading the Edit Form

Builds on: Byte 8

In plain terms:

The GET version of Edit() retrieves an existing application and puts its current values into an editable form.

The code:

```

[HttpGet("/applications/{id:int}/edit")]
public async Task<IActionResult> Edit(int id)
{
    var app = await _db.Applications.FindAsync(id);

    if (app is null)
    {
        return CustomNotFound();
    }

    return View("Edit", app);
}

```

What's happening:

The application is retrieved using its ID. If found, the existing object is sent to the Edit view, allowing the form to display the current company, role, status, dates, and other information.

Why it matters:

This separates **displaying the edit form** from **actually updating the database**.

Byte 10: Updating an Application

Builds on: Byte 9

In plain terms:

The POST version of Edit() receives the modified form, validates it, finds the original database record, copies the editable values across, and saves the changes.

The code:

```
var existing = await _db.Applications.FindAsync(id);
```

```
if (existing is null)
```

```
{  
    return CustomNotFound();  
}
```

```
existing.Company = application.Company.Trim();
```

```
existing.Role = application.Role.Trim();
```

```
existing.Status = application.Status;
```

```
existing.AppliedOn = application.AppliedOn;
```

```
existing.JobUrl = application.JobUrl;
```

```
existing.Notes = application.Notes;
```

```
await _db.SaveChangesAsync();
```

What's happening:

The submitted object is not directly saved. Instead, the controller finds the existing database record and updates its editable fields. SaveChangesAsync() then persists those changes.

Why it matters:

This implements the **Update** part of CRUD while keeping control over exactly which fields can be changed.

Byte 11: Deleting an Application

Builds on: Byte 10

In plain terms:

Delete() removes an application from the database. It only accepts a POST request, rather than allowing deletion through a simple URL visit.

The code:

```
[HttpPost("/applications/{id:int}/delete")]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Delete(int id)
{
    var app = await _db.Applications.FindAsync(id);

    if (app is null)
    {
        return CustomNotFound();
    }

    _db.Applications.Remove(app);
    await _db.SaveChangesAsync();

    return RedirectToAction(nameof(Index));
}
```

What's happening:

The application is first found by ID. Entity Framework marks it for deletion with Remove(), and SaveChangesAsync() executes the deletion. Afterwards, the user is redirected back to the application list.

Why it matters:

This implements the **Delete** part of CRUD while avoiding accidental deletion through a GET request.

Byte 12: Routing and Security

Builds on: Bytes 5–11

In plain terms:

Attributes above each method tell ASP.NET which URL and HTTP method should trigger that action. Anti-forgery validation protects POST operations against forged form submissions.

The code:

```
[HttpPost("/applications")]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Create(...)

[HttpPost("/applications/{id:int}/delete")]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Delete(int id)
```

What's happening:

HttpGet handles pages that retrieve/display information, while HttpPost handles operations that change data. ValidateAntiForgeryToken verifies that POST requests came from the application's legitimate forms.

Why it matters:

The routing structure keeps the application predictable, while anti-forgery tokens add protection to state-changing operations.

PUTTING IT TOGETHER

The ApplicationsController is essentially the **traffic manager** for the Placement Tracker. A browser request reaches a specific controller action based on its URL and HTTP method. The action validates or retrieves data through ApplicationDbContext, then either displays a Razor view, changes the database, or redirects the user. Together, the actions implement the complete **CRUD cycle: Create, Read, Update, Delete**. The helper methods handle common concerns such as optional fields and 404 responses, keeping the main actions cleaner.

[ValidateAntiForgeryToken] is used to protect your **POST requests from CSRF (Cross-Site Request Forgery) attacks**.

- It generates a security token when the form is displayed.
- The token is submitted along with the form.
- ASP.NET checks whether the token is valid when the POST request arrives.
- If the token is missing or invalid, the request is rejected.

For example:

```
[HttpPost("/applications")]
```

```
[ValidateAntiForgeryToken]
```

```
public async Task<IActionResult> Create(models.Application application)
```

Here, it protects the **Create** operation.

Simple example

Without protection, a malicious website could potentially trick a logged-in user into submitting:

```
POST /applications/{id}/delete
```

The anti-forgery token makes sure the request came through your legitimate form rather than an unauthorized/fake request.

In one line:

ValidateAntiForgeryToken ensures that state-changing form requests actually originated from your application.

ErrorController.cs

Byte 1: Purpose of the Error Controller

Builds on: None — starting point

In plain terms:

ErrorController handles error URLs and displays the application's custom error page. In this project, it is mainly used for **404 Not Found** errors.

The code:

```
public class ErrorController : Controller
{
}
```

What's happening:

The class inherits from ASP.NET Core's Controller, so it can respond to HTTP requests and return Razor views.

Why it matters:

Instead of showing ASP.NET's default error page, the application can show its own 404.cshtml page.

Byte 2: Defining the Error Routes

Builds on: Byte 1

In plain terms:

The controller accepts requests such as /Error/404. The {code:int} part means the error code must be an integer.

The code:

```
[Route("/Error/{code:int}")]
[Route("/Error")]
public IActionResult Index(int code = 404)
```

What's happening:

There are two possible routes:

/Error/404

/Error

If no error code is provided, code defaults to 404.

Why it matters:

It gives the application a dedicated route for displaying error pages and allows different HTTP status codes to be passed in.

Byte 3: Setting the HTTP Status Code

Builds on: Byte 2

In plain terms:

The controller makes sure the response actually contains the correct HTTP error status code.

The code:

```
public IActionResult Index(int code = 404)
{
    Response.StatusCode = code;
```

What's happening:

If the application receives /Error/404, code is 404, so:

```
Response.StatusCode = 404;
```

sets the HTTP response status to **404 Not Found**.

Why it matters:

Displaying an error-looking page isn't enough. The actual HTTP response should also tell the browser/server that the request failed.

Byte 4: Returning the Custom 404 View

Builds on: Byte 3

In plain terms:

After setting the status code, the controller loads the project's custom 404.cshtml page.

The code:

```
if (code == 404)
{
    return View("~/Views/Shared/404.cshtml");
}
```

What's happening:

When the status code is 404, ASP.NET renders:

Views

└─ Shared

└─ 404.cshtml

The Shared folder is commonly used for views that can be displayed from different parts of the application.

Why it matters:

Users see the custom 404 design instead of a raw or framework-generated error page.

Byte 5: The Final Fallback

Builds on: Byte 4

In plain terms:

The controller currently returns the same 404.cshtml view even when the error code isn't 404.

The code:

```
if (code == 404)
{
    return View("~/Views/Shared/404.cshtml");
}
```

```
return View("~/Views/Shared/404.cshtml");
```

What's happening:

The if condition doesn't currently change the final result. Both paths return exactly the same view.

So this could technically be simplified to:

```
Response.StatusCode = code;
return View("~/Views/Shared/404.cshtml");
```

Why it matters:

The current version works, but the if is redundant unless you plan to create different pages later, such as 403.cshtml for Forbidden or 500.cshtml for Server Error.

Byte 6: How Error Handling Connects to the Application

Builds on: Bytes 1–5

In plain terms:

The comment refers to `UseStatusCodePagesWithReExecute("/Error/{0}")`. This tells ASP.NET to redirect error responses internally to the `ErrorController`.

The code:

```
UseStatusCodePagesWithReExecute("/Error/{0}")
```

What's happening:

For example, if a request results in:

404

ASP.NET can re-execute the request using:

`/Error/404`

which is then handled by:

```
[Route("/Error/{code:int}")]
```

```
public IActionResult Index(int code = 404)
```

The controller sets the status code and displays the custom error view.

Why it matters:

It connects the application's general error-handling middleware with this controller, creating a consistent custom error page.

PUTTING IT TOGETHER

`ErrorController` is a small controller dedicated to handling error responses. ASP.NET sends an error code such as 404 to `/Error/404`, and the controller receives that code. It then sets the HTTP response status and renders the custom `404.cshtml` page. The `UseStatusCodePagesWithReExecute()` middleware connects normal application errors to this controller. Currently, all codes display the same 404 page, so the controller could be simplified unless additional error pages are planned.

ApplicationDbContext.cs

Byte 1: Purpose of ApplicationDbContext

Builds on: None — starting point

In plain terms:

ApplicationDbContext is the **bridge between the C# application and the database**. It uses Entity Framework Core (EF Core) to let the application work with SQLite using C# objects instead of writing SQL for every operation.

The code:

```
public class ApplicationDbContext : DbContext
{
}
```

What's happening:

DbContext is an EF Core class responsible for communicating with the database. By inheriting from it, ApplicationDbContext gets EF Core's database functionality.

Why it matters:

Without the DbContext, the controllers would not have the normal EF Core mechanism for reading, adding, updating, and deleting database records.

Byte 2: Connecting the DbContext to Configuration

Builds on: Byte 1

In plain terms:

The constructor receives the database configuration from ASP.NET Core and passes it to the parent DbContext.

The code:

```
public ApplicationDbContext(
    DbContextOptions<ApplicationDbContext> options)
    : base(options)
{
}
```

What's happening:

DbContextOptions contains information about how the database should be configured, such as which database provider to use. base(options) passes those settings to EF Core's DbContext.

Why it matters:

This allows the application to configure SQLite externally instead of hard-coding the database connection inside the context.

Byte 3: Representing the Applications Table

Builds on: Byte 2

In plain terms:

DbSet<Application> represents the collection of placement applications in the database. You can think of it roughly as the C# interface to the Applications table.

The code:

```
public DbSet<Models.Application> Applications
    => Set<Models.Application>();
```

What's happening:

DbSet allows EF Core to perform operations such as:

```
_db.Applications.Add(application);
```

or:

```
_db.Applications.FindAsync(id);
```

EF Core translates these operations into the appropriate database commands.

Why it matters:

This is what allows the ApplicationController to interact with the database through _db.Applications.

Byte 4: Configuring the Application Entity

Builds on: Byte 3

In plain terms:

OnModelCreating() defines rules for how the Application C# class should be represented in the database.

The code:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
```

```

{
    base.OnModelCreating(modelBuilder);

    modelBuilder.Entity<Models.Application>(entity =>
    {
        // database rules
    });
}

```

What's happening:

ModelBuilder is EF Core's tool for configuring entities. Here, the Application entity is selected and its database rules are defined.

Why it matters:

It gives you explicit control over database structure and validation rules instead of relying entirely on EF Core's defaults.

Byte 5: Defining the Primary Key

Builds on: Byte 4

In plain terms:

Every application needs a unique identifier. Here, Id is configured as the **primary key**.

The code:

```
entity.HasKey(a => a.Id);
```

What's happening:

EF Core is told that the Id property uniquely identifies each application record.

For example:

Id	Company
1	Google
2	Microsoft
3	Amazon

Why it matters:

The primary key allows EF Core to uniquely find, update, or delete a particular application.

Byte 6: Required Fields and Maximum Length

Builds on: Byte 5

In plain terms:

The context also defines which fields are required and how much text they can contain.

The code:

```
entity.Property(a => a.Company)
```

```
.IsRequired()
```

```
.HasMaxLength(100);
```

```
entity.Property(a => a.Role)
```

```
.IsRequired()
```

```
.HasMaxLength(100);
```

```
entity.Property(a => a.Status)
```

```
.IsRequired()
```

```
.HasMaxLength(20);
```

What's happening:

IsRequired() means the database expects a value. HasMaxLength() limits the amount of text stored in that field.

For example, Company can contain up to 100 characters, while Status can contain up to 20.

Why it matters:

These rules help prevent invalid or unnecessarily large data from being stored in the database.

Byte 7: Configuring Optional Notes

Builds on: Byte 6

In plain terms:

Notes is optional, but it still has a maximum length.

The code:

```
entity.Property(a => a.Notes)
```

```
.HasMaxLength(1000);
```

What's happening:

Unlike Company, Role, and Status, there is no `IsRequired()`. Therefore, Notes can be empty/null. However, when provided, it should not exceed 1000 characters.

Why it matters:

It allows users to leave notes blank while still preventing excessively large values.

PUTTING IT TOGETHER

`ApplicationDbContext` is the **database layer used by the Placement Tracker**. It inherits from EF Core's `DbContext`, receives database configuration, and exposes the `Applications` table through `DbSet`. `OnModelCreating()` then defines how the `Application` model should behave in the database. The controller uses `_db.Applications` to perform CRUD operations, while EF Core translates those C# operations into SQLite database operations.

Application.cs

Byte 1: Purpose of the Application Model

Builds on: None — starting point

In plain terms:

`Application` represents **one placement/internship application** in the system. Each object contains the information that will eventually be stored as one row in the SQLite database.

The code:

```
public class Application : IValidatableObject
{
    public int Id { get; set; }

    public string Company { get; set; } = string.Empty;
    public string Role { get; set; } = string.Empty;
}
```

What's happening:

The class defines the structure of an application. For example, one object could represent an application to Google for a Software Engineer role.

`IValidatableObject` is implemented because this model needs some **custom validation rules** involving multiple fields.

Why it matters:

This model is the central data structure shared between the forms, controller, validation system, and database.

Byte 2: Application ID

Builds on: Byte 1

In plain terms:

Id uniquely identifies each placement application.

The code:

```
public int Id { get; set; }
```

What's happening:

Each application receives an integer ID, such as:

1 → Google

2 → Microsoft

3 → Amazon

EF Core uses this as the primary key because ApplicationDbContext configured Id with HasKey().

Why it matters:

The controller uses the ID to find a specific application for viewing, editing, or deleting.

Byte 3: Company Validation

Builds on: Byte 2

In plain terms:

The Company field is required and must contain between 2 and 100 characters.

The code:

```
[Required(ErrorMessage = "Company is required.")]
```

```
[StringLength(100, MinimumLength = 2,
```

```
    ErrorMessage = "Company must be 2-100 characters.")]
```

```
public string Company { get; set; } = string.Empty;
```

What's happening:

[Required] prevents the field from being empty, while [StringLength] limits its size.

These are **Data Annotations** — attributes that describe validation rules directly on the model.

Why it matters:

Invalid company names can be caught before the data is saved to the database.

Byte 4: Role Validation

Builds on: Byte 3

In plain terms:

The job role follows the same basic validation rules as the company name.

The code:

```
[Required(ErrorMessage = "Role is required.")]  
[StringLength(100, MinimumLength = 2,  
    ErrorMessage = "Role must be 2-100 characters.")]  
public string Role { get; set; } = string.Empty;
```

What's happening:

The role cannot be missing, and its length must be between 2 and 100 characters.

For example:

"Software Engineer" → valid

"" → invalid

"A" → invalid

Why it matters:

It prevents meaningless or incomplete job-role data from entering the system.

Byte 5: Application Status

Builds on: Byte 4

In plain terms:

Every application has a status, such as Applied, Interview, or Rejected. The default status is Applied.

The code:

```
[Required(ErrorMessage = "Status is required.")]  
public string Status { get; set; } = "Applied";
```

```
public static readonly string[] AllowedStatuses =  
    new[] { "Wishlist", "Applied", "Interview", "Offer", "Rejected" };
```

What's happening:

The default status is set to "Applied". AllowedStatuses contains the only status values the application is supposed to accept.

Why it matters:

The status list gives the application a controlled set of states instead of allowing arbitrary values such as "Maybe" or "Random".

Byte 6: Application Date

Builds on: Byte 5

In plain terms:

AppliedOn stores the date when the user applied. It is nullable because a wishlist application may not have an application date yet.

The code:

```
[DataType(DataType.Date)]  
[Display(Name = "Applied On")]  
public DateTime? AppliedOn { get; set; }
```

What's happening:

DateTime? means the value can contain a date **or null**.

[Display] changes the label shown in the UI from AppliedOn to:

Applied On

Why it matters:

A wishlist entry doesn't necessarily have an application date, so the field needs to support an empty value.

Byte 7: Optional Job URL

Builds on: Byte 6

In plain terms:

JobUrl stores the link to the job posting. It is optional, but if provided, it must look like a valid URL.

The code:

```
[Url(ErrorMessage = "Please enter a valid URL (e.g. https://example.com/job).")]
```

```
[Display(Name = "Job URL")]
```

```
public string? JobUrl { get; set; }
```

What's happening:

string? means the field can be null. [Url] tells the validation system to check the format when a URL is provided.

Why it matters:

Users can leave the job URL blank while still preventing obviously invalid URLs from being accepted.

Byte 8: Optional Notes

Builds on: Byte 7

In plain terms:

Users can store additional information about an application in Notes, but the text cannot exceed 1000 characters.

The code:

```
[StringLength(1000,
```

```
    ErrorMessage = "Notes cannot be longer than 1000 characters.")]
```

```
public string? Notes { get; set; }
```

What's happening:

The ? makes the field optional, while [StringLength(1000)] limits its size.

Why it matters:

It allows flexible notes without allowing unnecessarily large text to be stored.

Byte 9: Tracking When the Application Was Created

Builds on: Byte 8

In plain terms:

CreatedAt records when the application object was created.

The code:

```
[Display(Name = "Created At")]
```

```
public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
```

What's happening:

When a new Application object is created, CreatedAt automatically receives the current UTC time.

Why it matters:

The system can know when an application was originally created without requiring the user to enter the timestamp manually.

Byte 10: Custom Validation with IValidatableObject

Builds on: Bytes 1–9

In plain terms:

Data annotations handle simple rules, but some rules depend on **multiple properties**. IValidatableObject allows the model to define these custom business rules in one Validate() method.

The code:

```
public IEnumerable<ValidationResult> Validate(
    ValidationContext validationContext)
{
    // custom validation rules
}
```

What's happening:

When ASP.NET validates the Application model, it also calls Validate(). The method can return one or more ValidationResult objects describing validation failures.

Why it matters:

Rules such as "AppliedOn is required unless Status is Wishlist" cannot be expressed cleanly with a simple [Required] attribute.

Byte 11: Validating the Status

Builds on: Byte 10

In plain terms:

The custom validation checks that the status actually belongs to the allowed status list.

The code:

```
if (!AllowedStatuses.Contains(Status))
```

```

{
    yield return new ValidationResult(
        "Status must be one of: Wishlist, Applied, Interview, Offer, Rejected.",
        new[] { nameof(Status) });
}

```

What's happening:

Contains() checks whether the submitted status exists in AllowedStatuses.

If it doesn't, ValidationResult creates an error associated with the Status field.

Why it matters:

[Required] only checks whether a status exists. It does **not** check whether the value is one of the permitted statuses.

Byte 12: Validating Company and Role Content

Builds on: Byte 11

In plain terms:

The custom validation also protects against values that technically exist but contain only spaces or are shorter than two meaningful characters.

The code:

```

if (string.IsNullOrWhiteSpace(Company) ||
    Company.Trim().Length < 2)
{
    yield return new ValidationResult(
        "Company must be 2-100 characters.",
        new[] { nameof(Company) });
}

```

What's happening:

IsNullOrWhiteSpace() catches values such as:

```

""
"  "

```

Trim() removes spaces from the beginning and end before checking the meaningful length.

The same type of validation is performed for Role.

Why it matters:

A field could technically pass some basic checks while still containing meaningless input such as several spaces.

Byte 13: Conditional Applied-Date Rule

Builds on: Byte 12

In plain terms:

The application date is required for every status **except Wishlist**. This is a business rule because the requirement depends on the value of another property.

The code:

```
if (Status != "Wishlist" && AppliedOn is null)
{
    yield return new ValidationResult(
        "Applied date is required unless status is Wishlist.",
        new[] { nameof(AppliedOn) });
}
```

What's happening:

The condition checks two things:

1. Status is not Wishlist.
2. AppliedOn has no value.

If both are true, validation fails.

For example:

Wishlist + no date → valid

Applied + no date → invalid

Interview + date → valid

Why it matters:

This ensures the stored application data follows the actual business logic of the Placement Tracker.

PUTTING IT TOGETHER

Application.cs defines what a placement application looks like and what makes its data valid. Data annotations handle straightforward rules such as required fields, string lengths, URLs,

and display names. `IValidatableObject` handles more specific business rules, such as allowed statuses and the conditional application-date requirement. When the controller receives a form, ASP.NET validates this model before allowing the controller to save it. Together, the model acts as both the **data structure and validation layer** for the Placement Tracker.

- `obj/` → **Intermediate build files**
 - Created by .NET when you build/run the project.
 - Contains temporary files, NuGet information, generated project files, compilation data, etc.
 - You normally **don't edit anything inside it**.
- `obj/Debug/net8.0/` → Build files specifically for:
 - Debug → development/debugging configuration.
 - `net8.0` → the project targets **.NET 8**.
 - So this means: **intermediate files generated while building your .NET 8 project in Debug mode**.

Think of it like this

Your actual code

↓

Controllers/

Models/

Data/

Views/

Program.cs

↓

.NET builds

↓

`obj/Debug/net8.0/` ← temporary/intermediate stuff

↓

`bin/Debug/net8.0/` ← final compiled output

So:

`obj/` → temporary build workspace

obj/Debug/net8.0/ → temporary files for .NET 8 Debug build

bin/Debug/net8.0/ → compiled application/dependencies

You don't need to study these folders for your project explanation. Focus on:

Controllers/

Data/

Models/

Views/

wwwroot/

PlacementTracker.Tests/

Program.cs

PlacementTracker.csproj

appsettings.json

And yes, **obj/ and bin/ can normally be deleted**; running/building the project will recreate them automatically.

ApplicationsControllerTests.cs

Byte 1: Purpose of the Test Class

Builds on: None — starting point

In plain terms:

ApplicationsControllerTests contains automated tests for ApplicationsController. It checks whether the controller correctly performs operations such as displaying, creating, editing, and deleting applications.

The code:

```
public class ApplicationsControllerTests
{
    // controller tests
}
```

What's happening:

Each method marked with [Fact] is an individual **xUnit test**. The tests use an EF Core **InMemory database**, meaning they test database behavior without touching the real SQLite file.

Why it matters:

Tests let you verify that changes to the controller don't accidentally break existing functionality.

Byte 2: Creating a Controller with a Test Database

Builds on: Byte 1

In plain terms:

The ControllerWithDb() helper creates an ApplicationsController using a database specifically created for testing.

The code:

```
private static ApplicationsController ControllerWithDb(
    ApplicationDbContext db)
{
    var controller = new ApplicationsController(db);
}
```

```

        controller.ControllerContext = new ControllerContext
        {
            HttpContext = new DefaultHttpContext()
        };

        return controller;
    }

```

What's happening:

The test database context is passed into the controller just like ASP.NET would normally provide it. A DefaultHttpContext is also attached because the controller's CustomNotFound() method accesses Response.StatusCode.

Why it matters:

It creates a realistic controller environment without starting the entire web application.

Byte 3: Testing the Dashboard

Builds on: Byte 2

In plain terms:

This test verifies that the dashboard retrieves all applications from the database.

The code:

```
db.Applications.Add(TestHelpers.ValidApplication());
```

```

db.Applications.Add(new Models.Application
{
    Company = "Northstar",
    Role = "Graduate Engineer",
    Status = "Interview"
});

```

```
await db.SaveChangesAsync();
```

```
var result = await controller.Index();
```

```
var view = Assert.IsType<ViewResult>(result);  
var model = Assert.IsAssignableFrom<List<Models.Application>>(view.Model);  
  
Assert.Equal(2, model.Count);
```

What's happening:

Two applications are inserted into the test database. Index() is called, and the test verifies that the returned view contains exactly two applications.

Why it matters:

It confirms that the controller's dashboard correctly retrieves database records.

Byte 4: Testing an Empty Database

Builds on: Byte 3

In plain terms:

This test checks the opposite situation: what happens when there are no applications.

The code:

```
using var db = TestHelpers.CreateDb();  
  
var controller = ControllerWithDb(db);  
var result = await controller.Index();  
  
var view = Assert.IsType<ViewResult>(result);  
var model = Assert.IsAssignableFrom<List<Models.Application>>(view.Model);  
  
Assert.Empty(model);
```

What's happening:

No records are added to the database. The test calls Index() and verifies that the returned list is empty.

Why it matters:

It confirms that an empty database doesn't cause an error and correctly produces an empty dashboard model.

Byte 5: Testing Successful Creation

Builds on: Byte 3

In plain terms:

This test checks the **Create** operation when valid application data is submitted.

The code:

```
var app = TestHelpers.ValidApplication();

var result = await controller.Create(app);

var redirect = Assert.IsType<RedirectToActionResult>(result);

Assert.Equal("Details", redirect.ActionName);
Assert.Equal(1, db.Applications.Count());
```

What's happening:

A valid application is submitted to `Create()`. The test expects the controller to save it and redirect to the Details action.

It also checks that exactly one record exists in the database.

Why it matters:

It verifies both important results of successful creation: **data is saved + the user is redirected correctly.**

Byte 6: Testing Invalid Creation

Builds on: Byte 5

In plain terms:

This test makes the application invalid and checks that the controller refuses to save it.

The code:

```
controller.ModelState.AddModelError(
    "Company",
    "Company is required."
);
```

```
app.Company = "";
```

```
var result = await controller.Create(app);
```

```
var view = Assert.IsType<ViewResult>(result);
```

```
Assert.Equal("Create", view.ViewName);
```

```
Assert.Equal(0, db.Applications.Count());
```

What's happening:

A validation error is manually added to ModelState. The controller receives the invalid application, detects that the model is invalid, returns the Create view, and does not save anything.

Why it matters:

It confirms that invalid user input doesn't reach the database.

Byte 7: Testing Application Details

Builds on: Byte 5

In plain terms:

This test checks whether the controller can retrieve and display one existing application.

The code:

```
db.Applications.Add(TestHelpers.ValidApplication());
```

```
await db.SaveChangesAsync();
```

```
var id = db.Applications.Single().Id;
```

```
var result = await controller.Details(id);
```

```
var view = Assert.IsType<ViewResult>(result);
```

```
var model = Assert.IsType<Models.Application>(view.Model);
```

```
Assert.Equal("Acme Labs", model.Company);
```

What's happening:

A record is inserted first. Its generated ID is retrieved, then Details(id) is called. The test verifies that the returned view contains the correct application.

Why it matters:

It confirms the controller's **single-record Read operation** works correctly.

Byte 8: Testing a Missing Application

Builds on: Byte 7

In plain terms:

This test checks what happens when someone requests an application ID that doesn't exist.

The code:

```
var result = await controller.Details(999999);
```

```
var view = Assert.IsType<ViewResult>(result);
```

```
Assert.Equal("~/Views/Shared/404.cshtml", view.ViewName);
```

```
Assert.Equal(404, controller.Response.StatusCode);
```

What's happening:

ID 999999 is intentionally unlikely to exist. The controller should return the custom 404 view and set the HTTP response status to 404.

Why it matters:

It verifies that missing records are handled gracefully instead of causing an exception or displaying incorrect data.

Byte 9: Testing the Edit Page

Builds on: Byte 8

In plain terms:

This test verifies that the edit page loads an existing application with its current values.

The code:

```
var result = await controller.Edit(id);
```

```
var view = Assert.IsType<ViewResult>(result);  
var model = Assert.IsType<Models.Application>(view.Model);
```

```
Assert.Equal(id, model.Id);  
Assert.Equal("Acme Labs", model.Company);
```

What's happening:

An existing application is retrieved using its ID. The test checks that the returned model contains the same ID and company information.

Why it matters:

It ensures the edit form can be correctly pre-filled with the existing application's data.

Byte 10: Testing an Update

Builds on: Byte 9

In plain terms:

This test verifies that editing an application actually updates the stored database record.

The code:

```
var updated = new Models.Application  
{  
    Company = "Contoso",  
    Role = "Offer Role",  
    Status = "Offer",  
    AppliedOn = new DateTime(2026, 9, 5),  
    JobUrl = "https://example.com/offer",  
    Notes = "Updated notes"  
};  
  
var result = await controller.Edit(id, updated);  
  
var redirect = Assert.IsType<RedirectToActionResult>(result);  
Assert.Equal("Details", redirect.ActionName);
```

What's happening:

New values are submitted to the POST version of Edit(). The test verifies that the controller redirects to Details and then checks the database to confirm the updated values were actually saved.

Why it matters:

It verifies the **Update** part of CRUD, including multiple editable fields rather than just one.

Byte 11: Testing Deletion

Builds on: Byte 10

In plain terms:

The final test verifies that an existing application can be deleted and that the controller redirects back to the dashboard.

The code:

```
var result = await controller.Delete(id);

var redirect = Assert.IsType<RedirectToActionResult>(result);

Assert.Equal("Index", redirect.ActionName);
Assert.Equal(0, db.Applications.Count());
```

What's happening:

The application is first inserted into the test database. Delete(id) is called, and the test checks two things: the controller redirects to Index, and the database now contains zero applications.

Why it matters:

It verifies the **Delete** part of CRUD and confirms that the record is actually removed.

PUTTING IT TOGETHER

ApplicationsControllerTests tests the controller without running the entire web application. Each test creates an isolated InMemory database, creates a controller connected to it, performs an operation, and checks the result. The tests cover the complete CRUD flow: dashboard/read, creation, details, update, and deletion, along with important edge cases such as invalid input, an empty database, and an unknown ID. Assert statements are what determine whether each test passes or fails. Together, these tests provide automated evidence that the ApplicationsController behaves as expected.

IntegrationTests.cs

Byte 1: Purpose of Integration Tests

Builds on: None — starting point

In plain terms:

IntegrationTests tests the Placement Tracker as a **complete web application**, rather than testing the controller by itself. It sends real HTTP requests such as GET / and checks the actual HTTP response and HTML returned.

The code:

```
public class IntegrationTests
    : IClassFixture<WebApplicationFactory<Program>>
{
    private readonly WebApplicationFactory<Program> _factory;
}
```

What's happening:

WebApplicationFactory<Program> creates a test version of the ASP.NET application. This allows the tests to run through the application's actual routing, controllers, middleware, views, and database configuration.

Why it matters:

Controller unit tests verify individual pieces, while integration tests verify that the **pieces work together**.

Byte 2: Creating the Test Application

Builds on: Byte 1

In plain terms:

The constructor customizes the test version of the application before any HTTP requests are made.

The code:

```
public IntegrationTests(
    WebApplicationFactory<Program> factory)
{
    _factory = factory.WithWebHostBuilder(builder =>
```

```
{  
    // test configuration  
});  
}
```

What's happening:

The supplied `WebApplicationFactory` is modified using `WithWebHostBuilder()`. This lets the tests change things like the environment and database without changing the real application configuration.

Why it matters:

The production application remains untouched while the tests get their own safe configuration.

Byte 3: Using the Testing Environment

Builds on: Byte 2

In plain terms:

The tests tell ASP.NET that the application is running in a special Testing environment.

The code:

```
builder.UseEnvironment("Testing");
```

What's happening:

Program.cs can check the current environment and behave differently when tests are running.

In this project, that prevents the tests from using the real SQLite database.

Why it matters:

Tests should not accidentally modify your real:

Data/placement_tracker.db

Byte 4: Replacing the Real Database

Builds on: Byte 3

In plain terms:

The integration tests replace the application's normal database configuration with an EF Core **InMemory database**.

The code:

```
services.AddDbContext<ApplicationDbContext>(options =>
```

```
options.UseInMemoryDatabase(  
    "IntegrationTests-" + Guid.NewGuid());
```

What's happening:

Instead of writing to SQLite, the test application stores its data in memory.

Guid.NewGuid() creates a unique database name for the test run, helping keep test data isolated.

Why it matters:

The integration tests can safely create, modify, and delete data without affecting your actual placement_tracker.db.

Byte 5: Removing the Existing Database Configuration

Builds on: Byte 4

In plain terms:

Before adding the InMemory database, the test removes the application's existing ApplicationDbContext registration.

The code:

```
var descriptor = services.SingleOrDefault(  
    d => d.ServiceType ==  
        typeof(DbContextOptions<ApplicationDbContext>));  
  
if (descriptor != null)  
{  
    services.Remove(descriptor);  
}
```

What's happening:

ASP.NET already has a database configuration registered. This code finds that registration and removes it.

The test can then register its own InMemory version.

Why it matters:

Without replacing the original configuration, the test could continue using the real SQLite database.

Byte 6: Creating an HTTP Client

Builds on: Bytes 1–5

In plain terms:

CreateClient() gives the test a tool that behaves like a browser making requests to the application.

The code:

```
var client = _factory.CreateClient();
```

```
var response = await client.GetAsync("/");
```

What's happening:

The test sends an actual HTTP GET request to /. The application processes it through its normal pipeline and returns an HTTP response.

Why it matters:

This tests more than just a controller method — it verifies that routing, middleware, controllers, database configuration, and views work together.

Byte 7: Testing the Empty Dashboard

Builds on: Byte 6

In plain terms:

This test checks that the home page works when there are no applications in the test database.

The code:

```
var response = await client.GetAsync("/");
```

```
Assert.Equal(HttpStatusCode.OK, response.StatusCode);
```

```
var html = await response.Content.ReadAsStringAsync();
```

```
Assert.Contains("No applications yet", html);
```

What's happening:

The test requests /, expects HTTP 200 OK, reads the returned HTML, and checks that the empty-state message appears.

Why it matters:

It verifies both the **HTTP response** and the **actual page content**, not just whether the controller returned successfully.

Byte 8: Testing the Create Form

Builds on: Byte 7

In plain terms:

This test verifies that a user can open the application creation page and that the expected form content is rendered.

The code:

```
var response =  
    await client.GetAsync("/applications/new");  
  
Assert.Equal(HttpStatusCode.OK, response.StatusCode);  
  
var html = await response.Content.ReadAsStringAsync();  
  
Assert.Contains("Add Application", html);  
Assert.Contains("Company", html);
```

What's happening:

The test requests `/applications/new`, checks for 200 OK, then searches the returned HTML for important text from the Create page.

Why it matters:

It catches problems where the route might exist but the expected form or UI content is missing.

Byte 9: Testing the Custom 404

Builds on: Byte 8

In plain terms:

This test checks the complete error-handling flow when a user requests an application that doesn't exist.

The code:

```
var response =
```

```
await client.GetAsync("/applications/999999");

Assert.Equal(
    HttpStatusCode.NotFound,
    response.StatusCode
);

var html = await response.Content.ReadAsStringAsync();
```

```
Assert.Contains("404", html);
```

What's happening:

The test requests a nonexistent application ID. It expects HTTP 404 Not Found and checks that the returned HTML contains 404.

This goes through the real routing → controller → error handling → view pipeline.

Why it matters:

It verifies that the application's custom 404 behavior works correctly from the user's perspective.

PUTTING IT TOGETHER

IntegrationTests runs the **actual ASP.NET application pipeline** in a controlled test environment. WebApplicationFactory creates the application, while the Testing environment and InMemory database keep it isolated from the real SQLite database. HttpClient then behaves like a browser and sends real HTTP requests to the application. The tests verify the HTTP status codes and HTML returned for the dashboard, create page, and missing application. So, unlike the earlier controller tests, these tests check whether the **whole application works together end-to-end at the HTTP level**.

TestHelpers.cs

Byte 1: Purpose of TestHelpers

Builds on: None — starting point

In plain terms:

TestHelpers contains reusable methods that make the automated tests easier to write. Instead of repeating database setup and sample application data in every test, the tests can call these helper methods.

The code:

```
public static class TestHelpers
{
    // reusable test methods
}
```

What's happening:

The class is static, meaning we don't need to create an object like `new TestHelpers()`. Its methods can be called directly:

```
TestHelpers.CreateDb();
```

```
TestHelpers.ValidApplication();
```

Why it matters:

It keeps the test files shorter and avoids duplicating the same setup code across many tests.

Byte 2: Creating an InMemory Database

Builds on: Byte 1

In plain terms:

`CreateDb()` creates a temporary EF Core database that exists only for testing. It does **not** use the project's real SQLite database.

The code:

```
public static ApplicationDbContext CreateDb()
{
    var options = new DbContextOptionsBuilder<ApplicationDbContext>()
        .UseInMemoryDatabase(
            databaseName: Guid.NewGuid().ToString())
        .Options;
```

What's happening:

DbContextOptionsBuilder creates the configuration for the test database.

UseInMemoryDatabase() tells EF Core to use an in-memory database instead of SQLite.

Guid.NewGuid() generates a unique name such as:

8f42c1...

Why it matters:

Tests can safely create and modify data without touching Data/placement_tracker.db.

Byte 3: Creating the Test DbContext

Builds on: Byte 2

In plain terms:

Once the database options are configured, the helper creates the same ApplicationDbContext used by the application, but with the test database configuration.

The code:

```
var db = new ApplicationDbContext(options);
```

```
db.Database.EnsureCreated();
```

```
return db;
```

What's happening:

The ApplicationDbContext receives the InMemory configuration. EnsureCreated() makes sure the database structure is ready before the test starts.

The completed context is then returned to the test.

Why it matters:

Tests can now use normal EF Core operations:

```
db.Applications.Add(...);
```

```
db.Applications.Count();
```

```
db.SaveChangesAsync();
```

without needing the real application database.

Byte 4: Creating Standard Test Data

Builds on: Byte 3

In plain terms:

ValidApplication() creates a ready-made placement application containing valid data. Tests can reuse it whenever they need a valid application.

The code:

```
public static Models.Application ValidApplication() => new()
{
    Company = "Acme Labs",
    Role = "Backend Intern",
    Status = "Applied",
    AppliedOn = new DateTime(2026, 9, 16),
    JobUrl = "https://example.com/job/123",
    Notes = "Test notes",
    CreatedAt = DateTime.UtcNow
};
```

What's happening:

The method uses C#'s object initializer syntax to create an Application with all the important fields already populated.

For example:

```
TestHelpers.ValidApplication()
```

returns an application that should pass the model's validation rules.

Why it matters:

Without this helper, every test that needs valid data would have to manually create the same Application object.

Byte 5: How the Helpers Are Used

Builds on: Bytes 1–4

In plain terms:

The test classes combine both helpers: one creates the test database, and the other creates valid application data.

The code:

```
using var db = TestHelpers.CreateDb();
```

```
db.Applications.Add(  
    TestHelpers.ValidApplication()  
);
```

```
await db.SaveChangesAsync();
```

What's happening:

The flow is:

CreateDb()



InMemory database



ValidApplication()



Valid Application object



Add + Save



Test data ready

Why it matters:

This gives every test a simple and consistent starting point while keeping tests isolated from the real database.

PUTTING IT TOGETHER

TestHelpers is a utility class created specifically to reduce repetition in the test project. CreateDb() provides an isolated EF Core InMemory database, while ValidApplication() provides a standard valid application object. The controller tests can combine these helpers to quickly prepare whatever data they need. Because each database receives a unique GUID-based name, the tests don't depend on the real SQLite file or on existing application data.

ValidationTests.cs

Byte 1: Purpose of Validation Tests

Builds on: None — starting point

In plain terms:

ValidationTests checks whether the Application model correctly accepts valid data and rejects invalid data. It focuses on rules such as required fields, character limits, allowed statuses, URLs, and conditional dates.

The code:

```
public class ValidationTests
{
    // validation tests
}
```

What's happening:

Each [Fact] method represents one specific validation scenario. Together, these tests verify the validation rules defined in Application.cs.

Why it matters:

It prevents invalid placement application data from being accepted when the model changes or new code is introduced.

Byte 2: Creating a Validation Helper

Builds on: Byte 1

In plain terms:

Instead of repeating the validation setup in every test, Validate() provides one reusable method for validating an Application.

The code:

```
private static IList<ValidationResult> Validate(
    Models.Application app)
{
    var context = new ValidationContext(
        app,
        mockServices.Object,
```

```

        null
    );

    var results = new List<ValidationResult>();

    Validator.TryValidateObject(
        app,
        context,
        results,
        validateAllProperties: true
    );

    return results;
}

```

What's happening:

Validator.TryValidateObject() runs the model's Data Annotation validation rules and collects any errors into results.

Because Application also implements IValidatableObject, its custom Validate() method is included in the validation process.

Why it matters:

Every test can simply call:

```
Validate(app)
```

and receive the list of validation errors.

Byte 3: Using Moq for the Validation Context

Builds on: Byte 2

In plain terms:

ValidationContext can accept an IServiceProvider, so the test creates a simple mocked version using **Moq**. The application doesn't actually need any services for these validation rules, but the context still expects one.

The code:

```
var mockServices = new Mock<IServiceProvider>();
```

```
mockServices
```

```
.Setup(s => s.GetService(It.IsAny<Type>()))  
.Returns((object?)null);
```

What's happening:

Moq creates a fake `IServiceProvider`. If validation asks for a service, the mock simply returns null.

Why it matters:

It allows the model to be validated in isolation without starting the entire ASP.NET dependency-injection system.

Byte 4: Testing Valid Data

Builds on: Bytes 2–3

In plain terms:

The first test confirms that a normal, valid application produces **zero validation errors**.

The code:

```
[Fact]  
public void Valid_application_passes_validation()  
{  
    var errors = Validate(  
        TestHelpers.ValidApplication()  
    );  
  
    Assert.Empty(errors);  
}
```

What's happening:

`ValidApplication()` provides known-good test data. `Validate()` checks it, and `Assert.Empty()` confirms that no errors were produced.

Why it matters:

This provides a baseline: valid application data should always pass validation.

Byte 5: Testing the Wishlist Date Rule

Builds on: Byte 4

In plain terms:

A Wishlist application is allowed to have no application date. This test specifically verifies that exception.

The code:

```
var app = TestHelpers.ValidApplication();
```

```
app.Status = "Wishlist";
```

```
app.AppliedOn = null;
```

```
Assert.Empty(Validate(app));
```

What's happening:

The test removes the applied date but changes the status to Wishlist. The model's business rule should allow this combination.

Why it matters:

It verifies the conditional rule from `Application.Validate()`:

Wishlist + no date → valid

Byte 6: Testing Required Company and Role

Builds on: Byte 4

In plain terms:

Company and role cannot be blank or contain only whitespace.

The code:

```
app.Company = "";
```

```
Assert.NotEmpty(Validate(app));
```

```
app.Role = "  ";
```

```
Assert.NotEmpty(Validate(app));
```

What's happening:

The tests deliberately provide invalid values and expect at least one validation error.

The role test uses spaces to make sure whitespace-only input is also rejected.

Why it matters:

It prevents meaningless application records such as an empty company or role from being accepted.

Byte 7: Testing Minimum Length

Builds on: Byte 6

In plain terms:

Company and role must contain at least two meaningful characters.

The code:

```
app.Company = "A";  
Assert.NotEmpty(Validate(app));  
  
app.Role = "x";  
Assert.NotEmpty(Validate(app));
```

What's happening:

Each test provides a one-character value. The [StringLength] rules in Application.cs require a minimum length of 2.

Why it matters:

It verifies that extremely short company or role names are rejected.

Byte 8: Testing Maximum Length

Builds on: Byte 7

In plain terms:

The tests also check the upper limits defined by the model: 100 characters for company and role, and 1000 characters for notes.

The code:

```
app.Company = new string('a', 101);  
Assert.NotEmpty(Validate(app));  
  
app.Role = new string('b', 101);  
Assert.NotEmpty(Validate(app));  
  
app.Notes = new string('n', 1001);  
Assert.NotEmpty(Validate(app));
```

What's happening:

`new string('a', 101)` creates a string containing 101 a characters. The tests deliberately go **one character beyond** each allowed limit.

Why it matters:

It verifies that the model's maximum-length validation actually works at the boundary.

Byte 9: Testing Allowed Statuses

Builds on: Byte 5

In plain terms:

The application status must be one of the predefined values. This test checks that an unknown status is rejected.

The code:

```
var app = TestHelpers.ValidApplication();
```

```
app.Status = "Hired";
```

```
Assert.NotEmpty(Validate(app));
```

What's happening:

"Hired" is not included in:

Wishlist

Applied

Interview

Offer

Rejected

Therefore, the custom `Validate()` method should produce an error.

Why it matters:

It prevents inconsistent status values from entering the application database.

Byte 10: Testing the Conditional Applied Date

Builds on: Byte 5

In plain terms:

If the status is anything other than Wishlist, an applied date must exist. This test checks that the correct field is reported as invalid.

The code:

```
app.Status = "Applied";
```

```
app.AppliedOn = null;
```

```
var errors = Validate(app);
```

```
Assert.Contains(  
    errors,  
    e => e.MemberNames.Contains("AppliedOn")  
);
```

What's happening:

The test creates an Applied application without a date. Instead of only checking that *some* error exists, it checks that the error specifically belongs to AppliedOn.

Why it matters:

This verifies not only that validation fails, but that the error is attached to the **correct form field**.

Byte 11: Testing Invalid URLs

Builds on: Byte 2

In plain terms:

JobUrl is optional, but when a URL is provided, it should have a valid URL format.

The code:

```
app.JobUrl = "not-a-url";
```

```
Assert.NotEmpty(Validate(app));
```

What's happening:

The [Url] Data Annotation on JobUrl checks the supplied value. The deliberately invalid string should generate a validation error.

Why it matters:

It prevents users from storing obviously invalid job links.

Byte 12: Testing Optional Fields

Builds on: Byte 11

In plain terms:

The final test confirms that optional fields can actually be left empty. `JobUrl` and `Notes` should accept null without producing validation errors.

The code:

```
app.JobUrl = null;
```

```
app.Notes = null;
```

```
Assert.Empty(Validate(app));
```

What's happening:

Both optional properties are explicitly set to null. Since neither property has `[Required]`, validation should still succeed.

Why it matters:

It confirms that the model distinguishes between **missing optional information** and **invalid information**.

PUTTING IT TOGETHER

`ValidationTests` verifies the validation rules defined in `Application.cs` without running the full web application. The `Validate()` helper runs both Data Annotation rules and the model's custom `IValidatableObject` rules. The tests then deliberately try valid data, missing values, incorrect lengths, invalid statuses, missing dates, bad URLs, and empty optional fields. Together, they test both normal behavior and important edge cases, helping ensure that only valid placement application data is accepted.

From your screenshot, these are the remaining **actual project files/folders** (not the generated bin/obj stuff):

- **Properties/**
 - Contains project-specific configuration files.
 - In ASP.NET Core, it commonly contains launchSettings.json.
 - launchSettings.json controls development settings such as the local URL/port and environment when running from VS Code/Visual Studio.
 - **You normally don't put application logic here.**
- **Views/**
 - Contains the **HTML/Razor pages** that users see.
 - Think of it as the **UI/frontend part** of your MVC application.
 - In your project:
 - Views/
 - |— Applications/
 - | |— Create.cshtml
 - | |— Details.cshtml
 - | |— Edit.cshtml
 - | |— Index.cshtml
 - |— Shared/
 - | |— 404.cshtml
 - |— _ViewImports.cshtml

└─ _ViewStart.cshtml

- **Views/Applications/**
 - Pages specifically related to placement applications.
 - Index.cshtml → dashboard/list
 - Create.cshtml → add application form
 - Details.cshtml → view one application
 - Edit.cshtml → edit application
- **Views/Shared/**
 - Contains views that can be used by different parts of the application.
 - Your 404.cshtml is the custom Not Found page.

- **_ViewImports.cshtml**
 - Provides common namespaces/tag helpers to Razor views.
 - Prevents you from having to repeatedly write the same `@using` statements in every `.cshtml` file.
- **_ViewStart.cshtml**
 - Defines common settings for Razor views.
 - Most importantly, it can specify the **layout** used by the pages.
- **wwwroot/**
 - Contains static frontend files.
 - Usually:
 - CSS
 - JavaScript

Images

- These files are sent directly to the browser.
- **appsettings.Development.json**
 - Configuration specifically for the **Development** environment.
 - Useful for development-specific settings.
- **appsettings.json**
 - Main application configuration file.
 - Can contain things like connection strings, logging settings, and application configuration.
- **PlacementTracker.csproj**
 - The **main project configuration file**.
 - Defines things such as:
 - .NET version
 - NuGet packages
 - project settings
 - dependencies
 - Example:

```
<TargetFramework>net8.0</TargetFramework>
```

- **Program.cs**

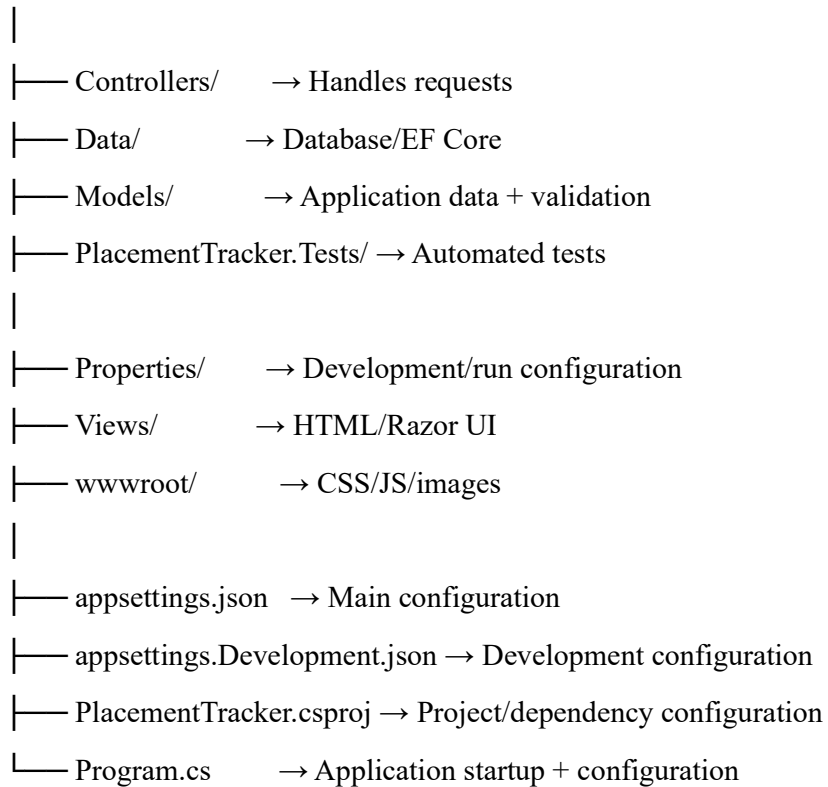
- The **starting point/configuration file** of your ASP.NET Core application.
- It configures services such as:
 - MVC
 - Entity Framework
 - SQLite
 - Dependency Injection
 - Middleware

Routing

- It also starts the web application.

The overall structure you're seeing

Placement Tracker - .NET



The easiest way to remember it:

Program.cs



Controllers



Models $\longleftrightarrow$ Data/Database



Views



Browser

And wwwroot provides the **CSS/JS/images** used by those Views.

Program.cs

Byte 1: Purpose of Program.cs

Builds on: None — starting point

In plain terms:

Program.cs is the **startup and configuration file** of the ASP.NET Core application. It sets up MVC, the database, error handling, routing, static files, and finally starts the web server.

The code:

```
var builder = WebApplication.CreateBuilder(args);
```

```
// Register application services here
```

```
var app = builder.Build();
```

```
// Configure the application here
```

```
app.Run();
```

What's happening:

The file has two major stages:

1. builder → configure services and application settings.
2. app → configure the HTTP request pipeline and start the application.

Why it matters:

Without Program.cs, ASP.NET doesn't know how your Placement Tracker should be configured or started.

Byte 2: Registering MVC and Razor Views

Builds on: Byte 1

In plain terms:

This tells ASP.NET that the project uses **MVC controllers and Razor views**.

The code:

```
builder.Services.AddControllersWithViews();
```

What's happening:

ASP.NET registers the services needed for:

Browser request



Controller



Razor View



HTML response

Why it matters:

Your ApplicationController and .cshtml files depend on these MVC services being registered.

Byte 3: Getting the Database Connection String

Builds on: Byte 2

In plain terms:

The application needs to know where its SQLite database is located. It first looks for a configured connection string and uses a default path if none is provided.

The code:

```
var connectionString =  
    builder.Configuration.GetConnectionString("Default")  
    ?? "Data Source=Data/placement_tracker.db";
```

What's happening:

The application looks for a connection string named Default.

If it doesn't exist, it uses:

Data/placement_tracker.db

This is the **persistent database** used by the actual application.

Why it matters:

This determines where placement application data is stored between application restarts.

Byte 4: Making Sure the Data Folder Exists

Builds on: Byte 3

In plain terms:

Before SQLite creates the database file, the application makes sure that the Data directory exists.

The code:

```
var dbFile = connectionString
    .Replace("Data Source=", "").Trim();

var dbDir = Path.GetDirectoryName(dbFile);

if (!string.IsNullOrEmpty(dbDir))
{
    Directory.CreateDirectory(dbDir);
}
```

What's happening:

The code extracts:

Data/placement_tracker.db

from the connection string, gets its directory (Data), and creates that directory if necessary.

Why it matters:

SQLite can create the database file, but the parent folder needs to exist first.

Byte 5: Registering Entity Framework Core

Builds on: Byte 3

In plain terms:

This connects ApplicationDbContext to SQLite using the connection string configured earlier.

The code:

```
builder.Services.AddDbContext<ApplicationDbContext>(
    options =>
        options.UseSqlite(connectionString));
```

What's happening:

ASP.NET's dependency injection system is told:

"Whenever something asks for ApplicationDbContext, provide one configured to use SQLite."

This is why the controller can receive:

```
public ApplicationsController(ApplicationDbContext db)
```

Why it matters:

It connects your controllers and EF Core to the actual SQLite database.

Byte 6: Building the Application

Builds on: Bytes 1–5

In plain terms:

After all the services have been configured, `builder.Build()` creates the actual ASP.NET application.

The code:

```
var app = builder.Build();
```

What's happening:

Everything configured through `builder.Services` is now used to construct the application.

After this point, `app` is used to configure how incoming HTTP requests should be processed.

Why it matters:

This is the transition from **configuration** to the actual running web application.

Byte 7: Development vs Production Errors

Builds on: Byte 6

In plain terms:

The application handles errors differently depending on whether it's running in development or production.

The code:

```
if (app.Environment.IsDevelopment())
{
    app.UseDeveloperExceptionPage();
}
```

```
else
{
    app.UseExceptionHandler("/Error/500");
}
```

What's happening:

During development, detailed error information is shown to help the developer debug problems.

In other environments, users are sent to:

/Error/500

instead of seeing internal error details.

Why it matters:

Developers need detailed errors, but exposing those details to normal users can reveal sensitive application information.

Byte 8: Handling 404 and Other Status Errors

Builds on: Byte 7

In plain terms:

This middleware allows the application to display its custom error page when a request produces an error status such as 404.

The code:

```
app.UseStatusCodePagesWithReExecute(
    "/Error/{0}");
```

What's happening:

If the application produces:

404

ASP.NET internally re-executes the request as:

/Error/404

which is handled by `ErrorController`.

Why it matters:

This connects the application's HTTP error responses to your custom error page.

Byte 9: Static Files and Routing

Builds on: Byte 8

In plain terms:

These middleware components allow the application to serve frontend files and determine which controller action should handle each request.

The code:

```
app.UseStaticFiles();  
app.UseRouting();  
app.UseAuthorization();
```

What's happening:

- `UseStaticFiles()` → serves CSS, JavaScript, images, etc. from `wwwroot`.
- `UseRouting()` → determines which endpoint/controller should handle a request.
- `UseAuthorization()` → enables authorization checks when the application uses them.

Why it matters:

These are key pieces of the HTTP request pipeline that allow the browser to communicate correctly with the application.

Byte 10: Default MVC Route

Builds on: Byte 9

In plain terms:

This defines the conventional URL pattern used by MVC when an attribute route isn't being used.

The code:

```
app.MapControllerRoute(  
    name: "default",  
    pattern: "{controller=Applications}" +  
        "{/action=Index}/{id?}");
```

What's happening:

The defaults mean:

Controller → Applications

Action → Index

ID → Optional

So a request to:

/

can reach:

ApplicationsController.Index()

Why it matters:

It provides the application's standard MVC routing fallback while your explicit [HttpGet(...)] and [HttpPost(...)] routes can take priority.

Byte 11: Creating the Database

Builds on: Byte 5

In plain terms:

When the application starts normally, this code makes sure the SQLite database and its tables exist.

The code:

```
if (!app.Environment.IsEnvironment("Testing"))
{
    using (var scope = app.Services.CreateScope())
    {
        var db = scope.ServiceProvider
            .GetRequiredService<ApplicationDbContext>();

        db.Database.EnsureCreated();
    }
}
```

What's happening:

The Testing environment is deliberately excluded.

For normal development/production environments, the application obtains ApplicationDbContext and calls EnsureCreated().

Why it matters:

The application can start with the required database structure without deleting and recreating the existing database.

Byte 12: Why Testing Skips the Real Database

Builds on: Byte 11

In plain terms:

The application deliberately avoids `placement_tracker.db` when running tests.

The code:

```
if (!app.Environment.IsEnvironment("Testing"))
{
    // create/use real SQLite database
}
```

What's happening:

Earlier, `IntegrationTests` configured:

```
builder.UseEnvironment("Testing");
```

Therefore this condition becomes false during integration tests.

The real SQLite database setup is skipped.

Why it matters:

Your automated tests cannot accidentally modify or delete your real placement data.

Byte 13: Development Sample Data

Builds on: Byte 11

In plain terms:

When running in Development, the application automatically inserts three sample applications if the database is empty.

The code:

```
if (app.Environment.IsDevelopment() &&
    !db.Applications.Any())
{
    db.Applications.AddRange(
        // 3 sample applications
    );
}
```

```
db.SaveChanges();  
}
```

What's happening:

Two conditions must be true:

Development environment

AND

Applications table is empty

Only then are the sample records inserted.

Why it matters:

It gives you realistic data to see when developing the UI without manually entering applications every time.

Byte 14: Starting the Web Application

Builds on: Bytes 1–13

In plain terms:

app.Run() starts the web application and makes it listen for incoming HTTP requests.

The code:

```
app.Run();
```

What's happening:

After all services, middleware, routes, database setup, and configuration have been prepared, ASP.NET starts the web server.

Your browser can then access the Placement Tracker.

Why it matters:

Without app.Run(), the application would be configured but would not actually start serving requests.

Byte 15: Making Program Visible to Integration Tests

Builds on: Byte 14

In plain terms:

The final line makes the Program class accessible to the integration testing project.

The code:

```
public partial class Program { }
```

What's happening:

WebApplicationFactory<Program> in your IntegrationTests.cs needs to locate the application's Program class.

This line provides an accessible partial Program type that the test infrastructure can reference.

Why it matters:

Without it, WebApplicationFactory<Program> may not be able to locate and create your ASP.NET application for integration testing.

PUTTING IT TOGETHER

Program.cs is essentially the **wiring and startup file** for the entire Placement Tracker. It first configures MVC and SQLite, then builds the application and sets up error handling, static files, routing, and authorization. On startup, it creates the real SQLite database when appropriate and optionally inserts development sample data. The Testing environment is specially handled so integration tests use their own InMemory database instead of the real SQLite file. Finally, app.Run() starts the application, while the Program declaration allows WebApplicationFactory to create the application during integration tests.

CONCEPTUAL NOTES

Yes. If your sir is doing a **conceptual viva of the complete .NET Placement Tracker**, he is unlikely to ask only “what does this line do?” He can ask **why you chose something, how components communicate, what happens internally, and how it differs from Java/Python.**

Based on your complete project structure and implementation, these are the questions I would prepare first.

1. Basic Project Architecture

Q1. What is your project?

Answer:

It is a Placement Tracker web application built using **ASP.NET Core MVC**. It allows users to create, view, edit, and delete placement or internship applications. It uses **Entity Framework Core** for database operations, **SQLite** for persistent storage, **Razor Views** for the frontend, and **xUnit** for automated testing.

Your project follows the general structure:

Browser



Controller



ApplicationDbContext



Entity Framework Core



SQLite

and the controller sends data to:

Controller



Razor View



HTML/CSS/JavaScript



Browser

Your project separates Controllers, Models, Data, Views, wwwroot, and Tests.

Q2. What does MVC stand for?

Answer:

MVC stands for Model-View-Controller.

- **Model** → represents application data and validation.
- **View** → represents the user interface.
- **Controller** → handles requests and coordinates between Model and View.

In your project:

Model → Models/Application.cs

View → Views/Applications/*.cshtml

Controller → Controllers/ApplicationsController.cs

Q3. Why do we use MVC?

Answer:

MVC separates responsibilities. The controller doesn't need to contain HTML, the view doesn't directly manipulate the database, and the model contains the application's data and validation rules.

This makes the application easier to maintain and test.

2. ASP.NET Core Questions

Q4. What is ASP.NET Core?

Answer:

ASP.NET Core is Microsoft's cross-platform framework for building web applications and APIs using .NET and languages such as C#.

Your application uses ASP.NET Core MVC to process HTTP requests and return Razor views.

Q5. What is .NET?

Answer:

.NET is Microsoft's software development platform/runtime and ecosystem used to build applications such as web applications, APIs, desktop applications and services.

ASP.NET Core is the web-development framework built on top of .NET.

Q6. What is C#?

Answer:

C# is the programming language used to implement this .NET application.

So:

C#

↓

.NET

↓

ASP.NET Core

↓

MVC Web Application

Q7. What is Program.cs?

Answer:

Program.cs is the entry point and startup configuration of the ASP.NET Core application.

It configures services such as MVC and Entity Framework Core, builds the application, configures middleware, defines routes, initializes the database, and finally starts the application.

3. Dependency Injection

Q8. What is Dependency Injection?

Answer:

Dependency Injection, or **DI**, is a design pattern where a class receives the objects it depends on from an external system instead of creating those objects itself.

Your controller receives:

```
public ApplicationsController(ApplicationDbContext db)
{
    _db = db;
}
```

ASP.NET Core creates the ApplicationDbContext and provides it to the controller.

Q9. Why didn't you simply do this?

```
var db = new ApplicationDbContext(...);
```

Answer:

Dependency Injection keeps object creation separate from business logic. It also makes testing easier because I can provide a different database context, such as an InMemory database, during tests.

Q10. Where is DI configured?

Answer:

In Program.cs:

```
builder.Services.AddDbContext<ApplicationDbContext>(  
    options => options.UseSqlite(connectionString));
```

This registers ApplicationDbContext with ASP.NET Core's dependency injection container.

4. Entity Framework Core

Q11. What is EF Core?

Answer:

EF Core stands for **Entity Framework Core**. It is Microsoft's Object-Relational Mapper, or ORM, for .NET.

It allows us to work with database records using C# objects rather than writing SQL for every operation.

Your ApplicationDbContext inherits from DbContext, which provides EF Core database functionality.

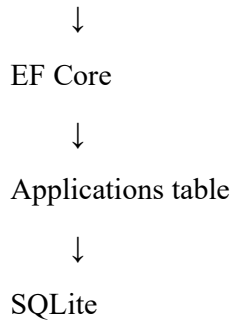
Q12. What is ORM?

Answer:

ORM stands for **Object-Relational Mapping**. It maps programming-language objects to relational database tables.

For your project:

C# Application object



Q13. What is DbContext?

Answer:

DbContext is EF Core's main class for communicating with the database. It manages querying, inserting, updating and deleting entities.

Your custom class:

ApplicationDbContext : DbContext

inherits these capabilities.

Q14. What is DbSet?

Answer:

DbSet represents a collection of entities that EF Core maps to a database table.

You have:

```
public DbSet<Models.Application> Applications  
    => Set<Models.Application>();
```

So Applications represents the application's database records.

Q15. What happens here?

```
_db.Applications.Add(application);  
await _db.SaveChangesAsync();
```

Answer:

Add() tells EF Core that a new entity should be inserted. SaveChangesAsync() actually persists that change to the database.

A very common viva follow-up is:

"Does Add() immediately insert into SQLite?"

Answer:

No. Add() marks the entity for insertion in the EF Core change tracker. SaveChangesAsync() sends the corresponding database operation.

5. SQLite

Q16. Why did you use SQLite?

Answer:

SQLite is a lightweight relational database that stores the database in a local file. It is suitable for this application because the Placement Tracker doesn't require a large database server.

Your connection is configured as:

Data Source=Data/placement_tracker.db

Q17. Is SQLite an in-memory database in your actual application?

Answer:

No. The actual application uses persistent SQLite storage. The InMemory database is only used during automated tests.

This distinction is **very important**.

Real application

→ SQLite

→ placement_tracker.db

Tests

→ EF Core InMemory

→ temporary test database

Your test setup deliberately avoids touching the real SQLite file.

6. Model Questions

Q18. What is Application.cs?

Answer:

It is the model/entity representing one placement or internship application.

For example:

Id

Company

Role

Status

AppliedOn

JobUrl

Notes

CreatedAt

One Application object corresponds conceptually to one database row.

Q19. Why does Application implement IValidatableObject?

Answer:

IValidatableObject allows us to define custom validation rules that cannot be easily represented using simple attributes.

For example:

Status = Wishlist

AppliedOn = null

→ Valid

Status = Applied

AppliedOn = null

→ Invalid

This requires checking **two properties together**.

7. Validation Questions

Q20. What is [Required]?

Answer:

[Required] ensures that a value must be provided.

Example:

[Required]

```
public string Company { get; set; }
```

Q21. What is [StringLength]?**Answer:**

[StringLength] restricts the length of a string.

Your company field uses:

```
[StringLength(100, MinimumLength = 2)]
```

So:

1 character → invalid

2–100 → valid

101+ → invalid

Q22. What is [Url]?**Answer:**

[Url] validates that the supplied value has a URL-like format.

You use it for JobUrl.

Q23. Why is JobUrl string??**Answer:**

The ? means the value is nullable. A user isn't required to provide a job URL.

Same idea for:

```
public string? Notes { get; set; }
```

Q24. Why isn't [Required] enough for Status?**Answer:**

Because [Required] only checks whether something was supplied.

It doesn't mean the value is valid.

For example:

```
Status = "Hired"
```

is not empty, but it isn't one of your allowed statuses.

Therefore you have custom validation:

AllowedStatuses.Contains(Status)

Your allowed statuses are Wishlist, Applied, Interview, Offer, and Rejected.

Q25. Why do you have both Data Annotations and custom validation?

Answer:

Data Annotations handle simple field-level rules, while IValidatableObject handles custom business rules.

Example:

[Required]

[StringLength]

[Url]

↓

Simple validation

IValidatableObject

↓

Business/conditional validation

8. CRUD Questions

Q26. What is CRUD?

Answer:

CRUD stands for **Create, Read, Update and Delete**.

Your project implements all four:

CRUD Your application

Create Add placement application

Read Dashboard/details

Update Edit application

Delete Delete application

The controller implements the complete CRUD cycle.

Q27. Which HTTP methods are you using?**Answer:**

Mainly GET and POST.

GET

→ retrieve/display information

POST

→ perform state-changing operations

For example:

```
[HttpGet("/applications/{id:int}")]
```

and:

```
[HttpPost("/applications")]
```

Q28. Why use POST for Delete instead of GET?**Answer:**

Because deletion changes server-side data. A GET request should generally be safe and not perform destructive operations.

Your delete route is:

```
[HttpPost("/applications/{id:int}/delete")]
```

and is protected with an anti-forgery token.

9. Routing**Q29. What is routing?****Answer:**

Routing determines which controller action handles a particular URL and HTTP method.

For example:

GET /

→ ApplicationsController.Index()

GET /applications/new

→ ApplicationsController.New()

GET /applications/5

→ ApplicationController.Details(5)

GET /applications/5/edit

→ ApplicationController.Edit(5)

Q30. What does {id:int} mean?

"/applications/{id:int}"

Answer:

It is a route constraint. It means that id must be an integer.

So:

/applications/5

matches.

But:

/applications/abc

doesn't match this route.

10. ModelState

Q31. What is ModelState?

Answer:

ModelState contains the results of model binding and validation for submitted request data.

Your Create action checks:

```
if (!ModelState.IsValid)
{
    return View("Create", application);
}
```

If invalid, the user gets the form back instead of the data being saved.

Q32. What is model binding?

Answer:

Model binding automatically converts incoming HTTP form values into C# objects.

For example, a form containing:

Company = Acme

Role = Backend Intern

Status = Applied

can be converted into:

Models.Application application

which is passed to your controller action.

11. Anti-Forgery / Security

Q33. What is CSRF?

Answer:

CSRF stands for **Cross-Site Request Forgery**. It is an attack where a malicious website attempts to make a user's browser perform an unwanted state-changing request to another website where the user is authenticated.

Q34. What does [ValidateAntiForgeryToken] do?

Answer:

It checks that a POST form submission contains a valid anti-forgery token generated for the application.

You use it on state-changing actions such as Create and Delete.

12. Razor Views / Frontend

Q35. Where is your frontend?

Answer:

The frontend is primarily implemented using **Razor Views**, which are .cshtml files inside the Views directory. CSS, JavaScript and images are served from wwwroot.

Your project structure explicitly separates Views for HTML/Razor UI and wwwroot for static assets.

Q36. What is Razor?

Answer:

Razor is the view-engine syntax used by ASP.NET Core to combine HTML with C# expressions.

The resulting page is rendered as HTML and sent to the browser.

Q37. What is .cshtml?

Answer:

.cshtml is the file extension used for Razor views. It allows HTML markup together with Razor/C# syntax.

13. Program.cs

Q38. What does AddControllersWithViews() do?

Answer:

It registers the services required for MVC controllers and Razor views.

```
builder.Services.AddControllersWithViews();
```

Q39. What is middleware?

Answer:

Middleware is software in the ASP.NET Core request pipeline that can inspect, modify, handle, or pass along HTTP requests and responses.

Your pipeline includes things such as:

```
app.UseStaticFiles();
```

```
app.UseRouting();
```

```
app.UseAuthorization();
```

Q40. What does UseStaticFiles() do?

Answer:

It allows ASP.NET Core to serve static files such as CSS, JavaScript and images from wwwroot.

Q41. What does UseRouting() do?

Answer:

It enables the routing system to determine which endpoint/controller should handle an incoming request.

Q42. What does `app.Run()` do?

Answer:

It starts the ASP.NET Core application's request-processing pipeline and begins listening for incoming requests.

14. Error Handling

Q43. Why do you have an `ErrorController`?

Answer:

It provides a centralized way to display custom error pages, particularly the custom 404 page.

Your application uses:

```
UseStatusCodePagesWithReExecute("/Error/{0}")
```

so an error such as 404 can be processed through `/Error/404`.

Q44. What is HTTP 404?

Answer:

HTTP 404 means **Not Found**. It means the requested resource could not be found.

For example:

```
/applications/999999
```

when application 999999 doesn't exist.

Q45. Why set `Response.StatusCode = 404` if you're already displaying a 404 page?

Answer:

Because the appearance of the page and the actual HTTP status are different things. The browser should receive the actual HTTP 404 status, not HTTP 200 with a page that merely looks like an error.

15. Database Initialization

Q46. What does `EnsureCreated()` do?

Answer:

EnsureCreated() checks whether the database exists and creates the database/schema if necessary.

It does not perform a normal migration-based schema evolution strategy.

Q47. Why do you skip database initialization in Testing?

Answer:

Because you don't want automated tests to touch:

Data/placement_tracker.db

Your Program checks:

```
if (!app.Environment.IsEnvironment("Testing"))
```

before creating/initializing the real database.

16. Development vs Testing

Q48. What is an environment in ASP.NET Core?

Answer:

An environment identifies how the application is currently running, such as Development, Production or Testing.

Your application behaves differently depending on the environment.

Development

→ developer error details

→ sample seed data

Testing

→ don't touch real SQLite database

→ tests use InMemory database

Q49. Why seed sample data only in Development?

Answer:

Sample records are useful while developing and demonstrating the application, but they should not automatically be inserted into a real production database.

17. Testing

Q50. What testing framework are you using?

Answer:

xUnit.

Individual tests are marked using:

[Fact]

Q51. What is a unit test?

Answer:

A unit test tests a relatively isolated piece of functionality.

Your ApplicationControllerTests directly create the controller and call methods such as:

controller.Index()

controller.Create(app)

controller.Edit(id)

controller.Delete(id)

They use an EF Core InMemory database rather than the real SQLite database.

Q52. What is an integration test?

Answer:

An integration test checks whether multiple parts of the application work together.

Your integration tests use:

WebApplicationFactory<Program>

and send actual HTTP requests using:

client.GetAsync("/")

So they test routing, middleware, controllers, views and the test database configuration together.

Q53. Difference between your controller test and integration test?

This is **very likely** to be asked.

Controller Test

Tests controller more directly

Directly calls controller method

Uses controller + DbContext

Doesn't need full application startup

Faster/more isolated

Example: `controller.Index()`

Integration Test

Tests complete application pipeline

Sends HTTP request

Uses `WebApplicationFactory`

Starts test version of application

More realistic

Example: `client.GetAsync("/")`

Q54. Why use an InMemory database in tests?

Answer:

To isolate tests from the real SQLite database and avoid modifying real application data.

Your test helper creates a unique database using:

`.UseInMemoryDatabase(Guid.NewGuid().ToString())`

18. TestHelpers

Q55. Why do you have TestHelpers.cs?

Answer:

It avoids duplicating common test setup.

For example:

`TestHelpers.CreateDb()`

`TestHelpers.ValidApplication()`

allow multiple tests to quickly create an isolated database and valid test data.

Q56. Why use Guid.NewGuid() for test databases?

Answer:

A GUID, or **Globally Unique Identifier**, provides a unique database name so separate tests don't unintentionally share the same InMemory database.

19. Validation Tests

Q57. What exactly are your validation tests testing?

Answer:

They test things like:

Company:

2–100 characters

Role:

2–100 characters

Status:

must be an allowed status

AppliedOn:

required unless Wishlist

JobUrl:

valid URL if supplied

Notes:

maximum 1000 characters

They also test edge cases such as blank/whitespace company and role.

Q58. Why test validation separately?

Answer:

Because validation is business logic. Separate tests make sure that invalid input is rejected even if the controller or UI changes later.

20. async / await

Q59. Why are many controller methods asynchronous?

Answer:

Database operations can take time. Using asynchronous methods such as `ToListAsync()`, `FindAsync()` and `SaveChangesAsync()` allows the application to avoid blocking the request thread while waiting for I/O.

Q60. What does await mean?

Answer:

`await` asynchronously waits for an operation to complete and then continues execution when the result is available.

21. IActionResult

Q61. Why does your controller return IActionResult?

Answer:

`IActionResult` allows an action to return different types of HTTP results.

For example, the same action might return:

`ViewResult`

`RedirectToActionResult`

404 response

This gives controller actions flexibility.

22. View() vs RedirectToAction()

Q62. Difference between View() and RedirectToAction()?

Answer:

`View()` directly renders a Razor view.

```
return View("Details", app);
```

`RedirectToAction()` tells the browser to make another request to another controller action.

```
return RedirectToAction(nameof(Details),
```

```
    new { id = application.Id });
```

A common pattern in your project is:

POST Create

↓

Save

↓

Redirect

↓

GET Details

23. nameof()

Q63. Why use nameof(Details) instead of "Details"?

Answer:

nameof() gets the name of the C# symbol at compile time. It reduces the chance of errors caused by manually typing strings.

nameof(Details)

produces:

"Details"

24. FindAsync()

Q64. What does FindAsync(id) do?

Answer:

It asks EF Core to find an entity by its primary key.

For example:

```
var app = await _db.Applications.FindAsync(id);
```

If it doesn't exist:

```
app == null
```

and your controller displays the custom 404 page.

25. Why Trim()?

Q65. Why do you use .Trim() for Company and Role?

Answer:

Trim() removes leading and trailing whitespace.

For example:

" Google "

becomes:

"Google"

This prevents unnecessary spaces from being stored.

26. DateTime.UtcNow

Q66. Why use DateTime.UtcNow for CreatedAt?

Answer:

UTC stands for **Coordinated Universal Time**. Storing timestamps in UTC avoids ambiguity caused by different time zones.

27. bin and obj

Q67. What is bin?

Answer:

bin contains compiled output generated by .NET, such as assemblies and dependencies.

Q68. What is obj?

Answer:

obj contains intermediate build files generated during compilation.

They are generated automatically and normally shouldn't be manually edited.

28. .csproj

Q69. What is .csproj?

Answer:

.csproj stands for **C# Project file**. It defines project configuration, target framework, package dependencies and other project settings.

29. Java Comparison

Since you implemented the same Placement Tracker in **Python, Java and .NET**, this is another area your sir can easily ask.

Q70. What is the .NET equivalent of Spring Boot?

Answer:

Conceptually:

Java	.NET
------	------

Spring Boot	↔	ASP.NET Core
Spring MVC	↔	ASP.NET Core MVC
Spring Data JPA	↔	Entity Framework Core
Hibernate	↔	EF Core's ORM functionality
Thymeleaf	↔	Razor Views
JUnit 5	↔	xUnit
Maven	↔	.NET project/SDK + NuGet ecosystem

Don't say they are **identical**. Say they provide broadly comparable roles.

Q71. What is the Python equivalent?

Your earlier Flask implementation maps roughly as:

Python Flask .NET

Flask	ASP.NET Core
Flask routes	Controller actions/routing
Jinja	Razor
Flask-SQLAlchemy	Entity Framework Core
SQLite	SQLite
pytest	xUnit

30. One Question Sir Can Absolutely Ask

Q72. Explain the complete flow when I add an application.

This is probably the **most important question to practice**.

Answer it like this:

First, the browser requests /applications/new. The controller's GET action creates a default Application object and returns the Create Razor view. The user fills in the form and submits it using POST. ASP.NET Core model binding converts the form fields into an Application object. Data annotations and custom validation are then checked through ModelState. If the model is invalid, the Create view is returned with the validation errors. If it is valid, the controller adds the object to ApplicationDbContext, calls SaveChangesAsync(), and EF Core stores it in SQLite. Finally, the controller redirects the user to the Details page.

The flow is:

Browser



GET /applications/new



ApplicationsController.New()



Create.cshtml



User submits form



POST /applications



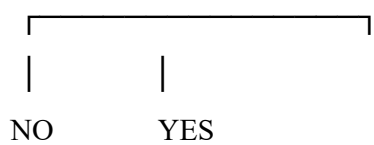
Model Binding



Validation



ModelState.IsValid?



Create View
again



DbContext



EF Core



SQLite



Redirect to Details

That explanation demonstrates that you understand the **whole architecture**, rather than memorizing individual files.

31. The 15 Questions I Would Memorize First

If your viva is soon, prioritize these:

1. **What is ASP.NET Core?**
2. **What is MVC?**
3. **Explain your project's architecture.**
4. **What is Dependency Injection?**
5. **What is DbContext?**
6. **What is DbSet?**
7. **What is Entity Framework Core?**
8. **What is ORM?**
9. **Explain CRUD in your project.**
10. **What is ModelState and model binding?**
11. **Why use Data Annotations and IValidatableObject?**
12. **Why use GET and POST differently?**
13. **What is [ValidateAntiForgeryToken] and CSRF?**
14. **Difference between unit testing and integration testing?**
15. **Explain the complete flow from browser → controller → database → view.**

And keep these full forms ready:

.NET → Microsoft's development platform
C# → C Sharp
MVC → Model View Controller
ASP → Active Server Pages
API → Application Programming Interface
ORM → Object-Relational Mapping
EF Core → Entity Framework Core
HTTP → Hypertext Transfer Protocol
GET → HTTP method for retrieving resources

POST → HTTP method commonly used for submitting/changing data
CRUD → Create Read Update Delete
CSRF → Cross-Site Request Forgery
URL → Uniform Resource Locator
HTML → HyperText Markup Language
CSS → Cascading Style Sheets
SQL → Structured Query Language
SQLite → Lightweight embedded relational database
DI → Dependency Injection
xUnit → .NET testing framework
GUID → Globally Unique Identifier
UTC → Coordinated Universal Time
UI → User Interface
Razor → ASP.NET Core view syntax/engine

One thing to be especially ready for: your sir may pick **any single line** and ask “*Why did you use this instead of something else?*” The strongest preparation is therefore not memorizing the code line-by-line, but understanding the chain:

Browser → HTTP → Routing → Controller → Model Binding → Validation → DbContext → EF Core → SQLite → Controller → Razor View → HTML → Browser.